Workgroup:	Network Working Group
Internet-Draft:	draft-narvaneni-agent-uri-04
Published:	20 October 2026
Intended Status:	Standards Track
Expires:	23 April 2027
Authors:	Y. Narvaneni *Independent Researcher*	S. P. Ravi *Independent Researcher*
---	---	---

The agent URI Scheme and Verifiable Agent Identity

Abstract

Software agents increasingly act on the Internet on behalf of people and organizations: they call APIs, browse websites, and delegate work to other agents. Protocols such as A2A and MCP define how agents communicate, but there is no interoperable way to answer a prior question: *which agent is this, who operates it, and how can its claims be verified?*

This document defines the agent URI scheme as a stable, human-readable identifier for an agent, a deterministic mapping from that identifier to an HTTPS-hosted Agent Identity Document, and a small set of mechanisms built on existing standards that let an agent prove its identity: signed statements binding the agent to its protocol metadata, an HTTP Message Signatures profile for requests made by the agent, short-lived credentials for ephemeral agent instances, and bindings to OAuth 2.0. It deliberately does not define how agents communicate, describe their capabilities, or are invoked.

Status of This Memo

This Internet-Draft is submitted in full conformance with the provisions of BCP 78 and BCP 79.

Internet-Drafts are working documents of the Internet Engineering Task Force (IETF). Note that other groups may also distribute working documents as Internet-Drafts. The list of current Internet-Drafts is at https://datatracker.ietf.org/drafts/current/.

Internet-Drafts are draft documents valid for a maximum of six months and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to use Internet-Drafts as reference material or to cite them other than as "work in progress."

This Internet-Draft will expire on 23 April 2027.

Copyright Notice

Copyright (c) 2026 IETF Trust and the persons identified as the document authors. All rights reserved.

This document is subject to BCP 78 and the IETF Trust's Legal Provisions Relating to IETF Documents (https://trustee.ietf.org/license-info) in effect on the date of publication of this document. Please review these documents carefully, as they describe your rights and restrictions with respect to this document. Code Components extracted from this document must include Revised BSD License text as described in Section 4.e of the Trust Legal Provisions and are provided without warranty as described in the Revised BSD License.

Table of Contents

1. Introduction 4
   1.1. Design Principles 4
   1.2. Non-Goals 4
2. Conventions and Terminology 5
3. Overview 5
4. The agent URI Scheme 6
   4.1. Syntax 6
   4.2. Normalization and Comparison 7
   4.3. Dereferencing 7
   4.4. Fragments 8
5. Agent Identity Document 8
   5.1. Format 8
   5.2. Keys 9
   5.3. Links 9
   5.4. Example 9
   5.5. Retrieval and Validation 10
6. Anchoring 11
   6.1. Origin-Anchored Agents 11
   6.2. DID-Anchored Agents 11
   6.3. Lifecycle, Rotation, and Revocation 11
7. Agent Statements 12
8. Proving Identity in Requests 12
   8.1. HTTP Message Signatures Profile 12
   8.2. Agent Instances 13

9. Authorization 14
9.1. The Agent as an OAuth Client 14
9.2. Delegation 14
9.3. Protected Resources 15
10. Discovery 15
11. Relationship to Other Protocols 15
12. Security Considerations 16
12.1. Trust Model 16
12.2. Handle Reassignment 16
12.3. Request-Destination Restrictions 16
12.4. Replay 16
12.5. Key Compromise 16
12.6. Cross-Origin Metadata 17
12.7. Confusable Names 17
13. Privacy Considerations 17
14. IANA Considerations 17
14.1. URI Scheme Registration 17
14.2. Well-Known URI Registration 18
14.3. Media Types 18
14.4. HTTP Field Names 18
14.5. Link Relations 18
14.6. OAuth Client Metadata 18
14.7. HTTP Message Signature Tag 19
15. References 19
15.1. Normative References 19
15.2. Informative References 20
Appendix A. Open Issues 21
Appendix B. Changes from draft-narvaneni-agent-uri-03 22
Acknowledgments 23
Authors' Addresses 23

1. Introduction

Agents are now first-class actors on the Internet. A single user task can cause an agent to call dozens of APIs and other agents, each of which needs to decide whether to serve the request. Today those services can see a TLS client, an IP address, perhaps an OAuth access token issued to *some* client, and a User-Agent string that anyone can forge. They cannot reliably tell which agent is calling, who is accountable for it, or whether the protocol metadata it advertises was really published by its operator.

Communication protocols already exist. [A2A] defines agent-to-agent interaction and an Agent Card; [MCP] defines how agents reach tools and publishes server metadata; OpenAPI describes HTTP APIs. Each of these describes *how to talk to* an endpoint. None of them defines a portable answer to *who* an agent is that works across all of them.

This document defines that missing identity layer:

1. **A name.** The `agent` URI (e.g., `agent://example.com/planner`) is a stable, human-readable identifier for an agent (Section 4).
2. **A document.** Every agent URI maps deterministically to an HTTPS URL that serves an Agent Identity Document listing the agent's operator, public keys, and links to its protocol metadata (Section 5).
3. **An anchor.** The document is trusted because of where it is served (the origin of the agent URI) or, for agents that need identity to survive a change of domain, because of a Decentralized Identifier (Section 6).
4. **Proofs.** Agents prove their identity by signing statements about their metadata (Section 7) and by signing the HTTP requests they make (Section 8), including from short-lived instances (Section 8.2). Identity is connected to OAuth 2.0 authorization without new token formats (Section 9).

1.1. Design Principles

- **Identify, do not invoke.** An agent URI names an agent. It is not a locator for an endpoint and carries no invocation parameters.
- **Reuse before invent.** Every mechanism is a profile of an existing standard: HTTPS and well-known URIs [RFC8615], JWK [RFC7517], JWS [RFC7515], HTTP Message Signatures [RFC9421], OAuth 2.0, and DIDs [DID-CORE].
- **Anchor trust outside the document being verified.** A key is trusted because of the origin or DID it is published under, never because a document names it about itself.
- **Separate the name from the long-lived identity when needed.** Domains change hands; agents get renamed and sold. The scheme supports a stable cryptographic identifier behind the human-readable name.
- **Work for agents that live for seconds.** Most agent executions are ephemeral. Instances act under the identity of their agent without being individually published.

1.2. Non-Goals

The following are explicitly out of scope:

- Communication semantics, message formats, task lifecycles, and invocation. These belong to A2A, MCP, OpenAPI-described APIs, and similar protocols.

- Capability, skill, or tool description. Protocol metadata documents (for example, an A2A Agent Card or an MCP server card) remain authoritative for these.
- Transport bindings.
- Search, ranking, or registry services. This document makes identity verifiable so that any registry can index agents without being trusted.
- Reputation, pricing, payments, and service levels.
- Authorization policy. Knowing who an agent is does not grant it anything.

2. Conventions and Terminology

The key words "**MUST**", "**MUST NOT**", "**REQUIRED**", "**SHALL**", "**SHALL NOT**", "**SHOULD**", "**SHOULD NOT**", "**RECOMMENDED**", "**NOT RECOMMENDED**", "**MAY**", and "**OPTIONAL**" in this document are to be interpreted as described in BCP 14 [RFC2119] [RFC8174] when, and only when, they appear in all capitals, as shown here.

This document uses the terms "JSON object", "member", and "string" from [RFC8259], "origin" from [RFC9110], and "structured field" from [RFC9651].

Agent: A software system that acts autonomously or semi-autonomously, typically on behalf of a person or organization.

Operator: The person or organization accountable for an agent.

Agent URI: An agent URI as defined in Section 4. The human-readable name of an agent.

Agent Identity Document (AID): A JSON document, retrieved by dereferencing an agent URI, that describes the agent's operator, keys, and links (Section 5).

Agent Key: A public key listed for an agent, used to verify statements, request signatures, and instance credentials made by the agent.

Instance: A running, typically short-lived execution of an agent that holds its own key pair and acts under the agent's identity (Section 8.2).

Verifier: Any party that checks an agent's identity: a website or API receiving requests, an authorization server, a registry, or another agent.

3. Overview

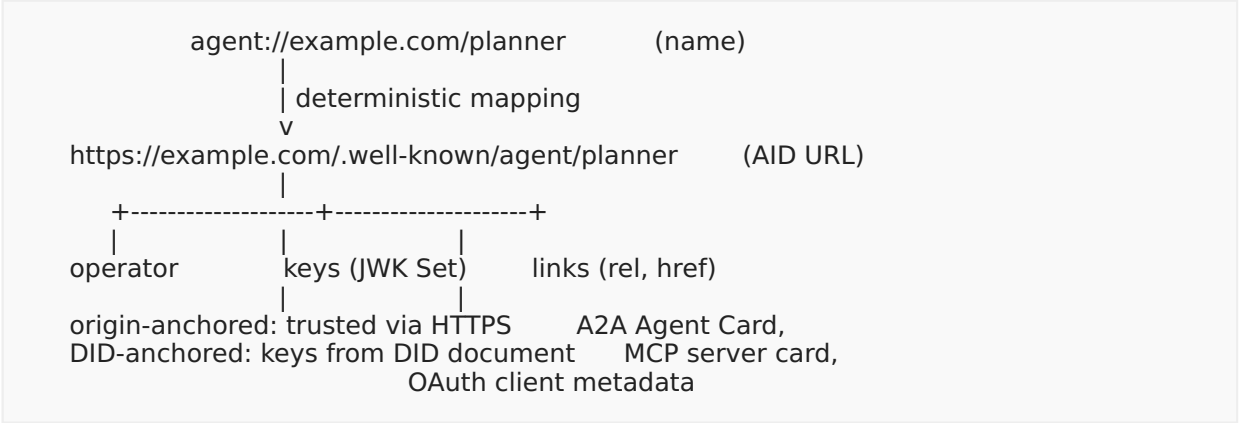


Figure 1: From agent URI to verifiable identity

A typical verification, for a request received by an API:

1. The request carries an HTTP message signature and a `Signature-Agent` field naming agent://example.com/planner.
2. The verifier maps the agent URI to its AID URL and retrieves the AID (or uses a cached copy).
3. The verifier finds the key identified by the signature, checks it is anchored (Section 6), and verifies the signature.
4. The verifier now knows which agent made the request and which operator is accountable. Whether to serve it remains a policy decision.

4. The agent URI Scheme

4.1. Syntax

The following ABNF [RFC5234] uses `port` and `fragment` from [RFC3986] and `ALPHA` and `DIGIT` from [RFC5234].

```
agent-uri    = "agent://" agent-host [ name-path ] [ "#" fragment ]

agent-host   = dns-name [ ":" port ]
dns-name     = label *( "." label )
label        = let-dig [ *61( let-dig / "-" ) let-dig ]
let-dig      = ALPHA / DIGIT

name-path    = 1*8( "/" name-segment )
name-segment = lc-let-dig [ *61( lc-let-dig / "-" ) lc-let-dig ]
lc-let-dig   = %x61-7A / DIGIT        ; a-z, 0-9
```

Figure 2: ABNF for the agent URI

Additional constraints:

- dns-name **MUST** be a fully qualified domain name. Internationalized domain names **MUST** be represented as A-labels [RFC5890] [RFC5891]. IP literals and userinfo are not permitted.
- An agent URI **MUST NOT** contain a query component.
- An agent URI with an empty name-path (e.g., agent://planner.example.com) identifies the domain's root agent.

Examples:

- agent://example.com/planner
- agent://planner.example.com
- agent://example.com/acme/billing-assistant
- agent://example.com/planner#k1 (a reference to key k1 of the agent; see Section 4.4)

4.2. Normalization and Comparison

The canonical form of an agent URI is produced by:

1. Lowercasing the scheme and dns-name.
2. Converting any U-labels to A-labels.
3. Removing a port equal to 443.
4. Removing a single trailing "/" if present.

Parsers **MAY** accept input that differs from the ABNF only by the deviations corrected above, and **MUST** normalize it. Any other deviation (uppercase in name-path, percent-encoding, dot-segments, a query) **MUST** be rejected.

Two agent URIs identify the same agent if and only if their canonical forms, with any fragment removed, are identical octet strings. In particular, agent://example.com/planner and agent://planner.example.com are different agents, even if the same operator runs both.

Agent URIs in AIDs, statements, signatures, and tokens **MUST** be in canonical form.

4.3. Dereferencing

An agent URI is dereferenced by mapping it to the HTTPS URL of its AID:

```
https://<agent-host>/.well-known/agent<name-path>      (named agent)
https://<agent-host>/.well-known/agent/_               (root agent)
```

For example:

Agent URI	AID URL
agent://example.com/planner	https://example.com/.well-known/agent/planner
agent://example.com/acme/billing-assistant	https://example.com/.well-known/agent/acme/billing-assistant

Agent URI	AID URL
agent://planner.example.com	https://planner.example.com/.well-known/agent/_
agent://example.com:8443/planner	https://example.com:8443/.well-known/agent/planner

Table 1: Agent URI to AID URL mapping

The root agent uses the reserved segment `_`, which cannot appear in a `name-segment`. This allows the root AID and named AIDs to coexist on static file hosting.

The mapping is purely syntactic. No registry or index needs to be fetched to dereference an agent URI.

4.4. Fragments

A fragment in an agent URI identifies an Agent Key by its `kid` (Section 5.2). For example, `agent://example.com/planner#k1` refers to the key with `"kid": "k1"` in the agent's key set. No other fragment semantics are defined.

5. Agent Identity Document

5.1. Format

An AID is a JSON object [RFC8259] served with media type `application/agent-id+json` (Section 14.3). It has the following members:

id: **REQUIRED**. String. The canonical agent URI that this document describes.

operator: **REQUIRED**. Object identifying who is accountable for the agent, with members `name` (**REQUIRED**, string) and `url` (**OPTIONAL**, string, HTTPS URL).

keys: **REQUIRED** for origin-anchored agents; **MUST** be absent for DID-anchored agents. A JWK Set [RFC7517] containing the agent's public keys (Section 5.2).

did: **OPTIONAL**. String. A DID [DID-CORE] whose DID document is the anchor for this agent's keys (Section 6.2).

name: **OPTIONAL**. String. A display name for people. It has no identity semantics.

status: **OPTIONAL**. String. One of `"active"` (default), `"suspended"`, or `"retired"` (Section 6.3).

links: **OPTIONAL**. Array of link objects (Section 5.3).

updated: **OPTIONAL**. String. The time the document was last changed, as an RFC 3339 timestamp.

Members not defined here **MUST** be ignored by verifiers that do not understand them. Extension member names **SHOULD** be namespaced (e.g., `"acme_tier"`) to avoid collision with future versions of this document.

An AID **MUST NOT** exceed 64 KiB.

5.2. Keys

Each key in keys **MUST**:

- contain only public key parameters;
- have a kid, which **SHOULD** be the JWK Thumbprint [RFC7638] of the key;
- have an alg member; and
- be usable for signatures ("use": "sig" or a key_ops containing "verify").

Implementations **MUST** support EdDSA (Ed25519) and ES256. Algorithms **MUST** be restricted as described in [RFC8725]; in particular, none and symmetric algorithms **MUST NOT** be accepted.

5.3. Links

Each link object has the following members, with the semantics of Web Linking [RFC8288]:

rel: **REQUIRED**. String. A registered link relation type or a URI.

href: **REQUIRED**. String. An absolute HTTPS URL.

type: **OPTIONAL**. String. The media type of the target.

statement: **OPTIONAL**. String. An Agent Statement (Section 7) about the target.

Link relations expected to be common include:

Relation	Target
agent-card	The agent's [A2A] Agent Card
mcp-server-card	The agent's [MCP] server metadata
service-desc	An OpenAPI or other API description [RFC9727]
oauth-client-metadata	The agent's OAuth Client ID Metadata Document (Section 9.1)

Table 2: Common link relations

A link to a target on the **same origin** as the agent URI is trusted to the same degree as the AID itself. A link to a target on **any other origin MUST** carry a statement; verifiers **MUST** ignore cross-origin links without a valid statement.

5.4. Example

```
{
 "id": "agent://example.com/planner",
 "name": "Example Trip Planner",
 "operator": {
  "name": "Example Travel Ltd",
  "url": "https://example.com"
 },
 "keys": {
  "keys": [
   {
    "kty": "OKP",
    "crv": "Ed25519",
    "x": "11qYAYKxCrfVS_7TyWQHOg7hcvPapiMlrwIaaPcHURo",
    "kid": "kPrK_qmxVWaYVA9wwBF6Iuo3vVzz7TxHCTwXBygrS4k",
    "alg": "EdDSA",
    "use": "sig"
   }
  ]
 },
 "links": [
  {
   "rel": "agent-card",
   "href": "https://example.com/planner/agent-card.json",
   "type": "application/json"
  },
  {
   "rel": "mcp-server-card",
   "href": "https://tools.example-cdn.net/planner/server.json",
   "statement": "eyJ0eXAiOiJhZ2VudC1zdGF0ZW1lbnQrand0Ii..."
  },
  {
   "rel": "oauth-client-metadata",
   "href": "https://example.com/planner/oauth-client.json"
  }
 ],
 "updated": "2026-10-20T09:00:00Z"
}
```

Figure 3: Example Agent Identity Document (origin-anchored)

5.5. Retrieval and Validation

To obtain the AID for an agent URI, a verifier:

1. Parses and canonicalizes the agent URI (Section 4.2), rejecting it on failure.
2. Constructs the AID URL (Section 4.3).
3. Applies the request-destination restrictions of Section 12.3.
4. Sends an HTTPS GET with `Accept: application/agent-id+json`, validating the server certificate for `dns-name`.
5. Follows redirects only to URLs with the same origin as the AID URL.
6. Rejects responses larger than 64 KiB, or that are not valid JSON objects. Verifiers **MUST NOT** reject a response solely because its media type is `application/json` (to support static hosting) but **MUST** reject any other media type.
7. Rejects the document unless its `id` member is identical to the canonical agent URI.

8. If `did` is present, performs the DID binding check of Section 6.2.

Verifiers **SHOULD** cache AIDs according to HTTP caching [RFC9111]. Operators **SHOULD NOT** set a freshness lifetime longer than 24 hours, so that key removal takes effect promptly (Section 6.3). Verifiers **SHOULD** cache failures for a short period to limit repeated fetches.

6. Anchoring

An Agent Key is *anchored* when the verifier has established that the agent's controller published it. This document defines two anchoring modes. An agent uses exactly one.

6.1. Origin-Anchored Agents

The AID contains `keys` and no `did`. The keys are anchored by the HTTPS retrieval of Section 5.5: control of the agent URI's origin is control of the agent's identity.

This mode needs nothing beyond a web server. Its weakness is that identity is exactly as durable as control of the domain: if the domain lapses or is transferred, so does the agent's identity (Section 12.2).

6.2. DID-Anchored Agents

The AID contains a `did` member and no `keys`. The agent's keys are the verification methods in the DID document's `assertionMethod` and `authentication` relationships [DID-CORE].

The binding between the agent URI and the DID **MUST** be bidirectional:

- the AID retrieved for the agent URI names the DID in `did`; and
- the DID document lists the canonical agent URI in its `alsoKnownAs` property.

Verifiers **MUST** check both directions and **MUST** treat the agent as unverified if either is missing.

DID-anchored agents can change their human-readable name without changing their identity: the operator publishes an AID under the new agent URI and adds the new URI to `alsoKnownAs`. Verifiers that record the DID (rather than only the agent URI) keep continuity across the rename.

DID methods that support verifiable key rotation history (for example, [DID-WEBVH]) are **RECOMMENDED**. Verifiers **MUST** maintain an allowlist of DID methods they support and **MUST** reject any other method.

6.3. Lifecycle, Rotation, and Revocation

- **Rotation.** Operators add the new key, start using it, and remove the old key once signatures made with it are no longer expected to be verified.
- **Revocation.** Removing a key from the AID (or DID document) revokes it. Verifiers learn of revocation when their cached copy expires.
- **Suspension and retirement.** If `status` is `"suspended"` or `"retired"`, verifiers **MUST** treat all signatures and credentials from the agent as invalid. A retired agent URI **SHOULD NOT** be reassigned to a different agent.

Operators that need faster or auditable revocation **MAY** additionally record their AIDs and statements in a transparency service ([SCITT-ARCH]). This document does not require it.

7. Agent Statements

An Agent Statement is a JWS [RFC7515] in compact serialization, signed with an Agent Key, that binds the agent to a specific version of a resource such as an Agent Card. It allows protocol metadata to be served from any origin, mirrored, or included in registries without losing its link to the agent.

The JWS Protected Header **MUST** contain:

- typ: "agent-statement+jwt";
- alg: the algorithm of the signing key;
- kid: the agent URI with a fragment identifying the signing key, e.g., "agent://example.com/planner#kPrK_qmxVWaYVA9wwBF6Iuo3vVzz7TxHCTwXBygrS4k".

The payload is a JWT Claims Set [RFC7519] with:

iss: **REQUIRED**. The canonical agent URI.

sub: **REQUIRED**. The URL of the resource the statement is about.

resource_digest: **REQUIRED**. An object with members alg ("sha-256"), canon ("jcs" or "none"), and value (the base64url-encoded digest). With "jcs", the digest is computed over the resource canonicalized per [RFC8785]; with "none", over the exact bytes of the representation.

iat: **REQUIRED**. Issue time.

exp: **OPTIONAL**. Expiry time. Statements about metadata that changes **SHOULD** expire.

To verify a statement, a verifier: checks typ; resolves iss to an AID; checks that the kid agent URI equals iss; locates and checks the anchoring of the key; verifies the signature; checks iat and exp; fetches the resource at sub (or uses a copy); and compares its digest with resource_digest.

Protocol metadata documents that support extension fields **SHOULD** include a reference back to the agent URI, so that a client starting from the metadata can find the identity. How that reference is expressed is defined by each protocol (see Appendix A).

8. Proving Identity in Requests

8.1. HTTP Message Signatures Profile

An agent proves its identity on an HTTP request by signing it with [RFC9421] as follows. The profile is designed to be compatible with the architecture of [WEB-BOT-AUTH].

The request **MUST** include a Signature-Agent header field whose value is a structured-field String [RFC9651] containing the canonical agent URI:

```
Signature-Agent: "agent://example.com/planner"
```

The signature **MUST**:

- cover at least @method, @authority, @target-uri, and signature-agent, and content-digest [RFC9530] when the request has content;
- include the created and expires parameters, with expires no more than 300 seconds after created;
- include a keyid parameter whose value is the JWK Thumbprint [RFC7638] of the signing key; and
- include the parameter tag="agent-uri".

A nonce parameter **SHOULD** be included where the verifier is expected to detect replay.

Example:

```
POST /v1/bookings HTTP/1.1
Host: api.example.net
Content-Type: application/json
Content-Digest: sha-256=:...:
Signature-Agent: "agent://example.com/planner"
Signature-Input: sig1=("@method" "@authority" "@target-uri"
    "signature-agent" "content-digest");created=1760950000;
    expires=1760950060;
    keyid="kPrK_qmxVWaYVA9wwBF6Iuo3vVzz7TxHCTwXBygrS4k";
    tag="agent-uri"
Signature: sig1=:...:
```

Figure 4: A signed request (line breaks for display only)

A verifier:

1. Parses Signature-Agent and canonicalizes the agent URI.
2. Selects the signature with tag="agent-uri".
3. Retrieves the AID (Section 5.5) and finds an anchored key whose thumbprint equals keyid, or, if an Agent-Instance field is present, validates the instance credential (Section 8.2).
4. Verifies the signature per [RFC9421] and checks that the current time is between created and expires, allowing for modest clock skew.
5. Checks status (Section 6.3).

On success, the verifier has established that the request was made by (an instance of) the agent. On failure, the verifier **MUST NOT** treat the request as coming from the named agent; it **MAY** process it as an unauthenticated request.

8.2. Agent Instances

Agents frequently run as many short-lived instances, for example one per task. Publishing a key for each instance in the AID does not scale, and keeping the agent's long-term private key in every instance is unsafe.

Instead, the agent (through its controlling infrastructure) issues each instance an **Agent Instance Credential**: a JWT [RFC7519] signed with an Agent Key, with:

- protected header typ: "agent-instance+jwt" and kid as in Section 7;
- iss: the canonical agent URI;
- sub: an identifier for the instance, unique per agent and **SHOULD** be random (Section 13);
- cnf: a confirmation claim containing the instance's public key as a jwk;
- iat and exp. The lifetime **SHOULD NOT** exceed one hour.

The instance signs requests with its own key as in Section 8.1, with keyid set to the thumbprint of the instance key, and sends the credential in the Agent-Instance header field as a structured-field String.

The verifier validates the credential against the agent's anchored keys, checks that iss equals the Signature-Agent value, and verifies the request signature using the key in cnf.

This construction is intentionally close to the Workload Identity Token of [WIMSE-S2S]; see Appendix A.

9. Authorization

Identity is not authorization. This section connects agent identity to OAuth 2.0 without defining new token formats.

9.1. The Agent as an OAuth Client

An agent that obtains OAuth tokens **SHOULD** use a Client ID Metadata Document [OAUTH-CIMD] as its client identifier and link to it from its AID with rel="oauth-client-metadata". The client metadata document **SHOULD** include the agent_uri client metadata parameter (Section 14.6) containing the canonical agent URI.

An authorization server that sees agent_uri **MAY** verify it by retrieving the AID and checking that it links to the same client_id. When verified, the authorization server knows which agent the client is and **MAY** show the agent's identity and operator to the user at consent time.

9.2. Delegation

When an agent acts on behalf of a user or another agent, the delegation **SHOULD** be expressed with OAuth 2.0 Token Exchange [RFC8693], with the agent identified in the act claim by "sub" set to its canonical agent URI. As required by [RFC8693], only the current actor is relevant to access-control decisions; nested actors are informational.

Access tokens used by agents **SHOULD** be sender-constrained, for example with DPoP [RFC9449] using the agent's or instance's key, so that a stolen token cannot be used by a different agent. Within a single trust domain, [TXN-TOKENS] can carry the agent identity and call context between services.

9.3. Protected Resources

Resource servers advertise their requirements using OAuth 2.0 Protected Resource Metadata [RFC9728]. A resource server **MAY** require a valid agent signature (Section 8.1) in addition to an access token, and **MAY** apply policy based on the agent URI, its operator, or its DID.

10. Discovery

Dereferencing a known agent URI requires no discovery. To let others find which agents a domain operates, a domain **MAY** list them in an API catalog [RFC9727] (a linkset [RFC9264]) at /.well-known/api-catalog, using the agent URI as the link context and agent-identity links to the AIDs:

```
{
 "linkset": [
   {
    "anchor": "agent://example.com/planner",
    "agent-identity": [
      { "href": "https://example.com/.well-known/agent/planner",
        "type": "application/agent-id+json" }
    ]
   }
 ]
}
```

Figure 5: Listing agents in an API catalog

Publishing a catalog is optional (Section 13). Registries and search services that index agents **SHOULD** verify AIDs and statements themselves rather than asking clients to trust the index.

11. Relationship to Other Protocols

A2A: An Agent Card describes how to communicate with an agent. The AID links to it with `rel="agent-card"`. A2A clients can verify a card by checking the agent's statement about it.

MCP: MCP server metadata describes tools and how to reach them. The AID links to it with `rel="mcp-server-card"`. MCP clients can use the agent URI and request signatures to identify the calling agent to MCP servers.

OpenAPI: The AID links to API descriptions with `rel="service-desc"` [RFC9727].

Web Bot Auth: [WEB-BOT-AUTH] lets automated clients sign requests and publish keys. This document adds a named, operator-accountable identity and an optional DID anchor on top of the same signature mechanism.

WIMSE: WIMSE addresses workload identity inside and across trust domains. Agent instance credentials (Section 8.2) address the same problem for agents; alignment is an open issue.

12. Security Considerations

12.1. Trust Model

For origin-anchored agents, the trust anchors are DNS and the Web PKI: anyone who can obtain a certificate for the agent URI's host and serve content there can impersonate the agent. This is the same trust model as the web itself and is appropriate for most agents. DID-anchored agents can reduce this dependency, at the cost of trusting the DID method's security.

12.2. Handle Reassignment

If a domain lapses and is registered by someone else, the new holder can publish an AID for the same agent URIs. Verifiers that record identities for audit or reputation **SHOULD** record the key thumbprint (and DID, when present) alongside the agent URI, and **SHOULD** treat a change of all keys for an agent as a significant event.

12.3. Request-Destination Restrictions

Signature-Agent values arrive in requests from untrusted parties, so any verifier can be made to fetch AIDs on attacker-chosen hosts. Verifiers **MUST**:

- refuse to fetch AID URLs whose host resolves to addresses in the IANA IPv4 and IPv6 Special-Purpose Address Registries that are not globally reachable, unless explicitly configured to allow them (e.g., for private deployments);
- apply the same check after every redirect and pin the resolved address for the duration of the fetch;
- enforce timeouts and the 64 KiB size limit; and
- rate-limit fetches per host and cache results, including failures.

12.4. Replay

Signatures cover the target authority and URI, which prevents replay against a different service. The short created-expires window limits replay against the same service; verifiers for non-idempotent operations **SHOULD** require and track nonce values.

12.5. Key Compromise

Compromise of an Agent Key lets the attacker act as the agent until the key is removed and caches expire. Operators **SHOULD** keep Agent Keys in hardware or a key management service and use instance credentials (Section 8.2) so that long-term keys are never present in running agents. Instance credentials **SHOULD** be short-lived, because they cannot be individually revoked.

12.6. Cross-Origin Metadata

Requiring statements for cross-origin links (Section 5.3) prevents a compromised or malicious third-party host from attaching metadata to an agent's identity. A statement does not vouch for the *contents* of the resource beyond its integrity: an agent can make false claims in its own Agent Card, and verifiers must not treat identity as an endorsement.

12.7. Confusable Names

Agent URIs can be visually similar (e.g., `examp1e.com`). User interfaces that display agent identities **SHOULD** display the operator name and the A-label form of the domain, and **SHOULD** warn when a DID-anchored agent's DID is new to the user.

13. Privacy Considerations

The considerations of [RFC6973] apply.

- **Linkability is the point, for agents.** Request signatures make an agent's traffic attributable and linkable across services. That is intended for the agent; it must not extend to the people it acts for. Agent URIs, instance identifiers, and `Signature-Agent` values **MUST NOT** encode user identifiers. Instance `sub` values **SHOULD** be random and not reused across users.
- **Per-user agents.** Deployments that create one agent per end user (e.g., `agent://example.com/u/alice`) make that user's activity linkable across all services the agent visits. Such deployments **SHOULD** use a shared agent identity and convey user context only through authorization tokens sent to parties entitled to it.
- **Enumeration.** API catalogs (Section 10) let anyone list a domain's agents. Domains that do not want their agents to be enumerable should not publish them in a catalog; AIDs remain retrievable by those who already know the agent URI.
- **Verifier fetches.** Fetching an AID reveals to the operator that a verifier has seen the agent. Caching reduces this signal.

14. IANA Considerations

14.1. URI Scheme Registration

This document requests registration of the `agent` URI scheme in the "Uniform Resource Identifier (URI) Schemes" registry per [RFC7595]:

- Scheme name: agent
- Status: Permanent
- Applications/protocols that use this scheme name: Identification of software agents and retrieval of their Agent Identity Documents.
- Contact: The authors.
- Change controller: IETF.
- Reference: Section 4 of this document.

This document does not define or request registration of agent+<suffix> schemes.

14.2. Well-Known URI Registration

This document requests registration in the "Well-Known URIs" registry [RFC8615]:

- URI suffix: agent
- Change controller: IETF
- Specification document: Section 4.3 of this document
- Status: permanent
- Related information: Sub-paths are used; see Section 4.3.

14.3. Media Types

This document requests registration of the following media types per [RFC6838]:

- `application/agent-id+json`, for Agent Identity Documents (Section 5).
- `application/agent-statement+jwt`, for Agent Statements (Section 7).
- `application/agent-instance+jwt`, for Agent Instance Credentials (Section 8.2).

Full registration templates will be provided in a future revision.

14.4. HTTP Field Names

This document requests registration in the "Hypertext Transfer Protocol (HTTP) Field Name Registry":

Field Name	Status	Structured Type	Reference
Signature-Agent	permanent	Item	Section 8.1
Agent-Instance	permanent	Item	Section 8.2

Table 3: HTTP field registrations

If `Signature-Agent` is registered by [WEB-BOT-AUTH], this document will instead reference that registration and define agent URIs as a permitted value.

14.5. Link Relations

This document requests registration of `agent-identity` in the "Link Relation Types" registry [RFC8288], with description "Refers to the Agent Identity Document of the link context." Registration of agent-card, mcp-server-card, and oauth-client-metadata is an open issue (Appendix A).

14.6. OAuth Client Metadata

This document requests registration of `agent_uri` in the "OAuth Dynamic Client Registration Metadata" registry [RFC7591], with description "Canonical agent URI of the client".

14.7. HTTP Message Signature Tag

This document uses the signature tag value agent-uri. Whether a registry entry is needed is an open issue.

15. References

15.1. Normative References

[RFC2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, DOI 10.17487/RFC2119, March 1997, <https://www.rfc-editor.org/rfc/rfc2119>.

[RFC3986] Berners-Lee, T., Fielding, R., and L. Masinter, "Uniform Resource Identifier (URI): Generic Syntax", STD 66, RFC 3986, DOI 10.17487/RFC3986, January 2005, <https://www.rfc-editor.org/rfc/rfc3986>.

[RFC5234] Crocker, D., Ed. and P. Overell, "Augmented BNF for Syntax Specifications: ABNF", STD 68, RFC 5234, DOI 10.17487/RFC5234, January 2008, <https://www.rfc-editor.org/rfc/rfc5234>.

[RFC5890] Klensin, J., "Internationalized Domain Names for Applications (IDNA): Definitions and Document Framework", RFC 5890, DOI 10.17487/RFC5890, August 2010, <https://www.rfc-editor.org/rfc/rfc5890>.

[RFC5891] Klensin, J., "Internationalized Domain Names in Applications (IDNA): Protocol", RFC 5891, DOI 10.17487/RFC5891, August 2010, <https://www.rfc-editor.org/rfc/rfc5891>.

[RFC6838] Freed, N., Klensin, J., and T. Hansen, "Media Type Specifications and Registration Procedures", BCP 13, RFC 6838, DOI 10.17487/RFC6838, January 2013, <https://www.rfc-editor.org/rfc/rfc6838>.

[RFC7515] Jones, M., Bradley, J., and N. Sakimura, "JSON Web Signature (JWS)", RFC 7515, DOI 10.17487/RFC7515, May 2015, <https://www.rfc-editor.org/rfc/rfc7515>.

[RFC7517] Jones, M., "JSON Web Key (JWK)", RFC 7517, DOI 10.17487/RFC7517, May 2015, <https://www.rfc-editor.org/rfc/rfc7517>.

[RFC7519] Jones, M., Bradley, J., and N. Sakimura, "JSON Web Token (JWT)", RFC 7519, DOI 10.17487/RFC7519, May 2015, <https://www.rfc-editor.org/rfc/rfc7519>.

[RFC7591] Richer, J., Ed., Jones, M., Bradley, J., Machulak, M., and P. Hunt, "OAuth 2.0 Dynamic Client Registration Protocol", RFC 7591, DOI 10.17487/RFC7591, July 2015, <https://www.rfc-editor.org/rfc/rfc7591>.

[RFC7595] Thaler, D., Ed., Hansen, T., and T. Hardie, "Guidelines and Registration Procedures for URI Schemes", BCP 35, RFC 7595, DOI 10.17487/RFC7595, June 2015, <https://www.rfc-editor.org/rfc/rfc7595>.

[RFC7638] Jones, M. and N. Sakimura, "JSON Web Key (JWK) Thumbprint", RFC 7638, DOI 10.17487/RFC7638, September 2015, <https://www.rfc-editor.org/rfc/rfc7638>.

[RFC8174] Leiba, B., "Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words", BCP 14, RFC 8174, DOI 10.17487/RFC8174, May 2017, <https://www.rfc-editor.org/rfc/rfc8174>.

[RFC8259] Bray, T., Ed., "The JavaScript Object Notation (JSON) Data Interchange Format", STD 90, RFC 8259, DOI 10.17487/RFC8259, December 2017, <https://www.rfc-editor.org/rfc/rfc8259>.

[RFC8288] Nottingham, M., "Web Linking", RFC 8288, DOI 10.17487/RFC8288, October 2017, <https://www.rfc-editor.org/rfc/rfc8288>.

[RFC8615] Nottingham, M., "Well-Known Uniform Resource Identifiers (URIs)", RFC 8615, DOI 10.17487/RFC8615, May 2019, <https://www.rfc-editor.org/rfc/rfc8615>.

[RFC8725] Sheffer, Y., Hardt, D., and M. Jones, "JSON Web Token Best Current Practices", BCP 225, RFC 8725, DOI 10.17487/RFC8725, February 2020, <https://www.rfc-editor.org/rfc/rfc8725>.

[RFC8785] Rundgren, A., Jordan, B., and S. Erdtman, "JSON Canonicalization Scheme (JCS)", RFC 8785, DOI 10.17487/RFC8785, June 2020, <https://www.rfc-editor.org/rfc/rfc8785>.

[RFC9110] Fielding, R., Ed., Nottingham, M., Ed., and J. Reschke, Ed., "HTTP Semantics", STD 97, RFC 9110, DOI 10.17487/RFC9110, June 2022, <https://www.rfc-editor.org/rfc/rfc9110>.

[RFC9111] Fielding, R., Ed., Nottingham, M., Ed., and J. Reschke, Ed., "HTTP Caching", STD 98, RFC 9111, DOI 10.17487/RFC9111, June 2022, <https://www.rfc-editor.org/rfc/rfc9111>.

[RFC9421] Backman, A., Ed., Richer, J., Ed., and M. Sporny, "HTTP Message Signatures", RFC 9421, DOI 10.17487/RFC9421, February 2024, <https://www.rfc-editor.org/rfc/rfc9421>.

[RFC9530] Polli, R. and L. Pardue, "Digest Fields", RFC 9530, DOI 10.17487/RFC9530, February 2024, <https://www.rfc-editor.org/rfc/rfc9530>.

[RFC9651] Nottingham, M. and P. Kamp, "Structured Field Values for HTTP", RFC 9651, DOI 10.17487/RFC9651, September 2024, <https://www.rfc-editor.org/rfc/rfc9651>.

15.2. Informative References

[A2A] A2A Project, Linux Foundation, "Agent2Agent (A2A) Protocol Specification", 2025, <https://a2a-protocol.org/latest/specification/>.

[DID-CORE] Sporny, M., Longley, D., Sabadello, M., Reed, D., Steele, O., and C. Allen, "Decentralized Identifiers (DIDs) v1.0", W3C Recommendation, July 2022, <https://www.w3.org/TR/did-core/>.

[DID-WEBVH] Decentralized Identity Foundation, "The did:webvh DID Method", 2025, <https://identity.foundation/didwebvh/>.

[MCP] Model Context Protocol Project, "Model Context Protocol Specification", 2025, <https://modelcontextprotocol.io/specification/>.

[OAUTH-CIMD] Parecki, A. and E. Smith, "OAuth Client ID Metadata Document", 2025, <https://datatracker.ietf.org/doc/draft-ietf-oauth-client-id-metadata-document/>.

[RFC6973] Cooper, A., Tschofenig, H., Aboba, B., Peterson, J., Morris, J., Hansen, M., and R. Smith, "Privacy Considerations for Internet Protocols", RFC 6973, DOI 10.17487/RFC6973, July 2013, <https://www.rfc-editor.org/rfc/rfc6973>.

[RFC8693] Jones, M., Nadalin, A., Campbell, B., Ed., Bradley, J., and C. Mortimore, "OAuth 2.0 Token Exchange", RFC 8693, DOI 10.17487/RFC8693, January 2020, <https://www.rfc-editor.org/rfc/rfc8693>.

[RFC9264] Wilde, E. and H. Van de Sompel, "Linkset: Media Types and a Link Relation Type for Link Sets", RFC 9264, DOI 10.17487/RFC9264, July 2022, <https://www.rfc-editor.org/rfc/rfc9264>.

[RFC9449] Fett, D., Campbell, B., Bradley, J., Lodderstedt, T., Jones, M., and D. Waite, "OAuth 2.0 Demonstrating Proof of Possession (DPoP)", RFC 9449, DOI 10.17487/RFC9449, September 2023, <https://www.rfc-editor.org/rfc/rfc9449>.

[RFC9727] Smith, K., "api-catalog: A Well-Known URI and Link Relation to Help Discovery of APIs", RFC 9727, DOI 10.17487/RFC9727, June 2025, <https://www.rfc-editor.org/rfc/rfc9727>.

[RFC9728] Jones, M.B., Hunt, P., and A. Parecki, "OAuth 2.0 Protected Resource Metadata", RFC 9728, DOI 10.17487/RFC9728, April 2025, <https://www.rfc-editor.org/rfc/rfc9728>.

[SCITT-ARCH] Birkholz, H., Delignat-Lavaud, A., Fournet, C., Deshpande, Y., and S. Lasker, "An Architecture for Trustworthy and Transparent Digital Supply Chains", 2025, <https://datatracker.ietf.org/doc/draft-ietf-scitt-architecture/>.

[TXN-TOKENS] Tulshibagwale, A., Fletcher, G., and P. Kasselman, "Transaction Tokens", 2025, <https://datatracker.ietf.org/doc/draft-ietf-oauth-transaction-tokens/>.

[WEB-BOT-AUTH] Meunier, T., "HTTP Message Signatures for Automated Traffic Architecture", 2025, <https://datatracker.ietf.org/doc/draft-meunier-web-bot-auth-architecture/>.

[WIMSE-S2S] Campbell, B., Salowey, J., Schwenkschuster, A., and Y. Sheffer, "WIMSE Workload-to-Workload Authentication", 2025, <https://datatracker.ietf.org/doc/draft-ietf-wimse-s2s-protocol/>.

Appendix A. Open Issues

This appendix will be removed before publication.

1. **Web Bot Auth alignment.** [WEB-BOT-AUTH] uses `Signature-Agent` to point at a key directory URL. Decide whether agent URIs become a permitted value of that field, whether the AID also serves as (or links to) an HTTP message signatures directory, or whether a distinct field is needed.

2. **WIMSE alignment.** Decide whether Agent Instance Credentials should be a profile of the WIMSE Workload Identity Token [WIMSE-S2S] with iss set to the agent URI, rather than a separate type.
3. **Back-references from protocol metadata.** Agree with the A2A and MCP projects on how an Agent Card or MCP server card names its agent URI, and who registers the agent-card and mcp-server-card link relations.
4. **Reserved root segment.** /.well-known/agent/_ is chosen to work on static hosting. Alternatives (a separate well-known suffix for the root agent) should be evaluated.
5. **Name path depth and character set.** Lowercase LDH segments, at most 8, are chosen for unambiguous comparison. Confirm this is sufficient for multi-tenant platforms.
6. **DNS-based verification.** Consider an alternative proof of control via a DNS TXT record (as for some handle systems), for operators that cannot serve well-known paths.
7. **Revocation latency.** Evaluate whether cache lifetimes alone are adequate, or whether a status list or transparency log should be **RECOMMENDED**.
8. **Port.** Confirm that non-default ports should be permitted in agent identities at all, given they complicate certificate and policy handling.

Appendix B. Changes from draft-narvaneni-agent-uri-03

This revision changes the scope of the document from "addressing and discovery" to "identity".

Removed:

- agent+<protocol> transport-binding schemes and the Transport Bindings registry.
- Invocation semantics (GET/POST rules, fallback behaviour), query parameters in agent URIs, and conformance levels.
- The /.well-known/agents.json registry, replaced by the deterministic mapping of Section 4.3 and optional RFC 9727 catalogs.
- The agent descriptor's capability fields (skills, transport, interactionModel, versioning, content negotiation). Protocol metadata documents are linked instead.
- DID authorities in agent URIs, replaced by DID anchoring (Section 6.2).
- Cross-cutting conventions (tracing, idempotency, rate limits, health) and the Interaction Models registry.
- The monotonic scope narrowing rule, which conflicted with [RFC8693].

Added:

- Strict URI syntax, normalization, and comparison rules.
- The Agent Identity Document and anchoring modes.
- Agent Statements, the HTTP Message Signatures profile, and Agent Instance Credentials.
- OAuth client and delegation bindings.
- Security and privacy considerations specific to agent identity.

Acknowledgments

The authors thank the reviewers of earlier revisions, whose feedback led to the change of scope in this revision.

Authors' Addresses

Yaswanth Narvaneni
Independent Researcher
London
United Kingdom
Email: yaswanth+ietf@gmail.com

Sai Purnima Ravi
Independent Researcher
Email: r.sai.purnima+ietf@gmail.com